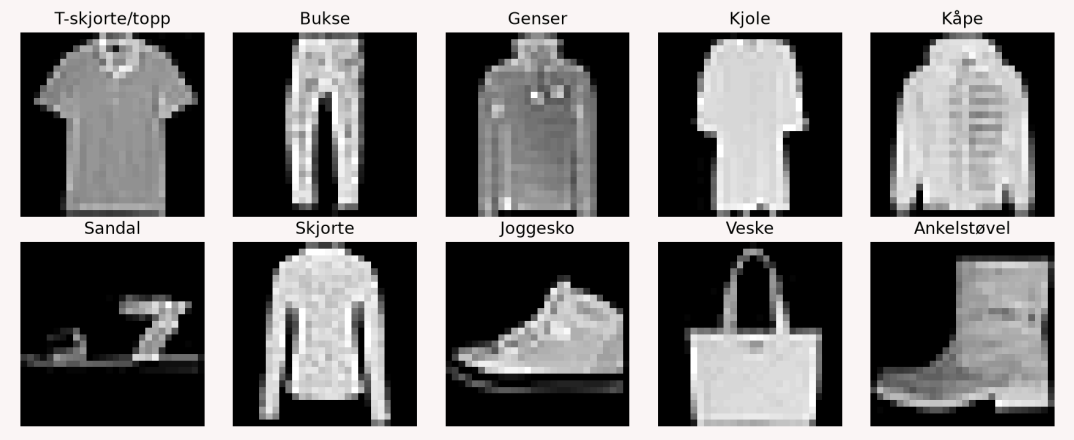
PyTorch 101

Tensorer, nevrale nettverk og bildeklassifikasjon

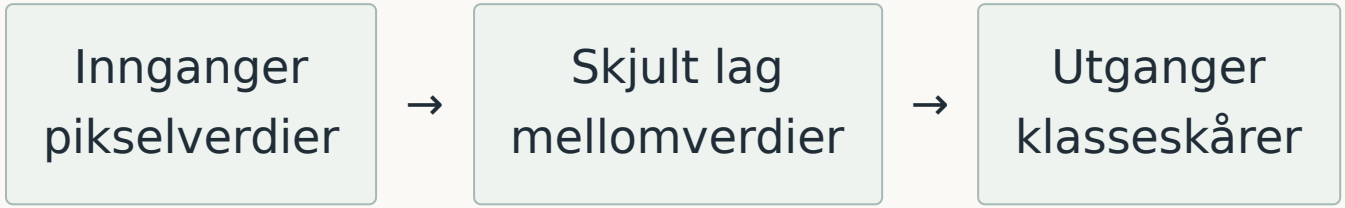
Bildeklassifikasjon



Bilde → Modell → Ti klasseskårer → Predikert klasse

Fashion-MNIST · Zalando Research

Nevrale nettverk

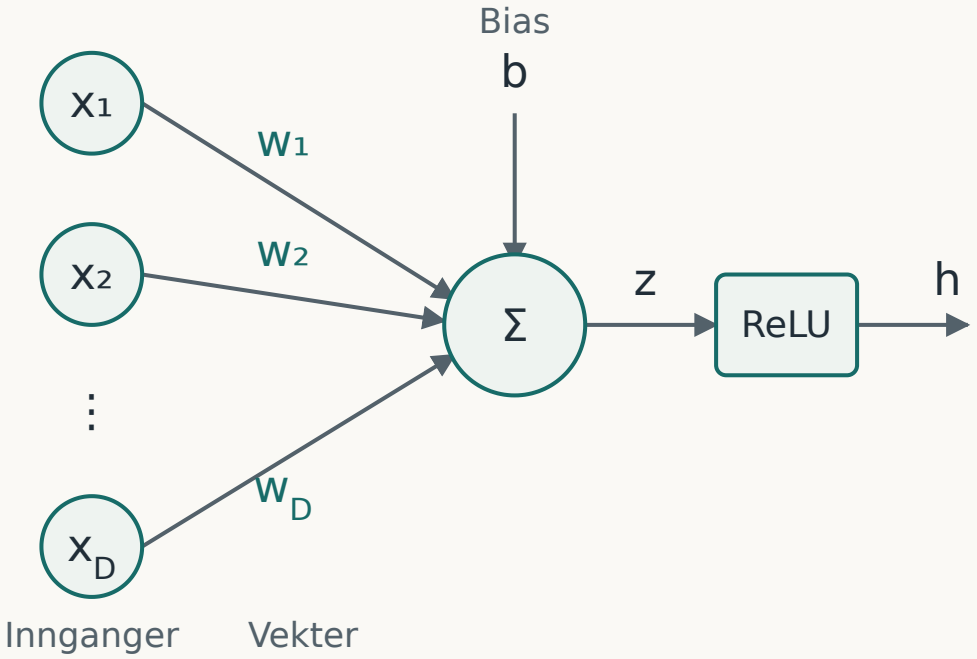


Et nevralt nettverk består av lag med lærbare parametere: **vekter og bias**.

Ulineære aktiveringsfunksjoner mellom lagene gjør at nettverket kan representere ulineære sammenhenger.

> PyTorch gir oss tensorer, modellag og automatisk beregning av gradienter.

Ett nevron



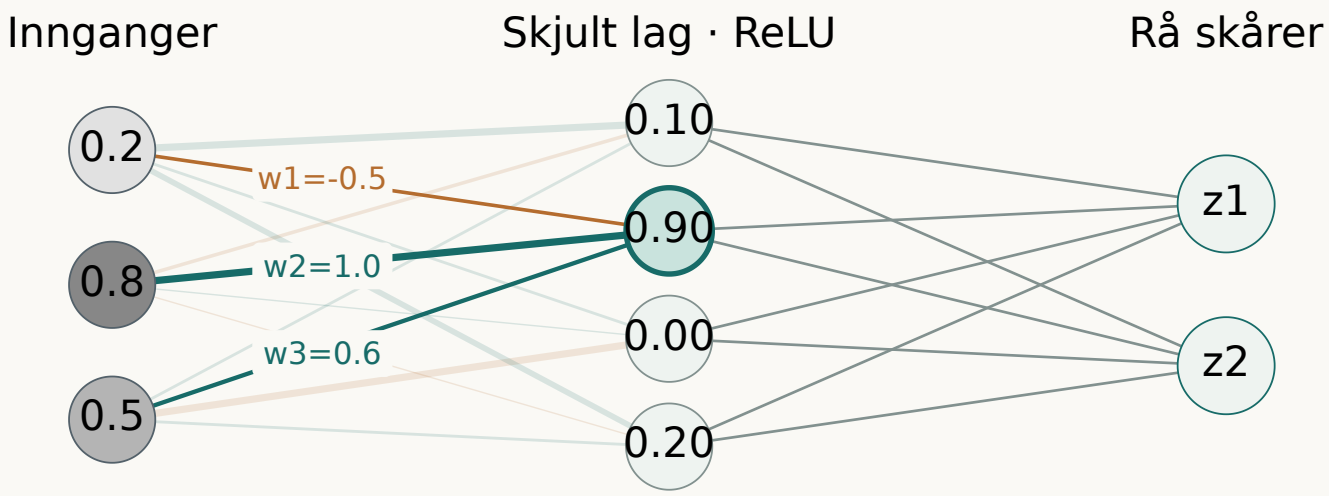
Vektet sum + bias

$$z = \sum_{i=1}^{D} w_i x_i + b = \mathbf{w}^\mathsf{T}\mathbf{x} + b$$

Aktivering

$$h = \mathrm{ReLU}(z) = \max(0, z)$$

Eksempel



Nevron: 1 2 3 4 Bias b = -0.1

$$z = (-0.5) \times 0.2 + (1.0) \times 0.8 + (0.6) \times 0.5 + (-0.1) = 0.90$$

$$h = \mathrm{ReLU}(z) = \max(0, 0.90) = 0.90$$

Tensorer og former

Tensor	Eksempel på form	Betydning
Skalar	`[]`	Ett tall
Vektor	`[3]`	Tre tall langs én akse
Matrise	`[2, 3]`	To rader, tre kolonner
Bildebatch	`[B, C, H, W]`	Bilder, kanaler, høyde, bredde

```
import torch
values = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
values.shape  # torch.Size([2, 3])
```

Indeksering og akser

```
values = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
values[0]                  # Første rad: [1., 2., 3.]
values[:, 1]               # Andre kolonne: [2., 5.]
values.mean(dim=1)         # Ett gjennomsnitt per rad: [2., 5.]
values + 2                 # Legg 2 til hvert tall
```

`dim` velger aksen vi regner over.

`shape`: organisering · `dtype`: talltype · `device`: CPU eller GPU

Matrisemultiplikasjon

$$C = AB, \qquad C_{ik} = \sum_{j=1}^{D} A_{ij} B_{jk}$$

$$[B, D] \mathbin{@} [D, K] \longrightarrow [B, K]$$

Elementet C_{ik} er prikkproduktet av rad i i A og kolonne k i B.

```
result = inputs @ weights
```

Matrisemultiplikasjon: eksempel

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 8 \\ 13 & 17 \end{bmatrix}$$

Første tall: $1 \cdot 2 + 2 \cdot 1 + 3 \cdot 0 = 4$.

```
inputs = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
weights = torch.tensor([[2., 0.], [1., 1.], [0., 2.]])
result = inputs @ weights  # Form [2, 2]
```

`@` beregner matriseprodukt. `*` ganger elementvis.

Oppgaver 01 · Tensorer

`oppgaver/01_tensorer.ipynb`

Indeksering, reduksjoner, broadcasting og matrisemultiplikasjon.

Kontrollspørsmål · Tensorer

`scores` har formen `[32, 10]`: 32 bilder og 10 klasseskårer per bilde.

```
predictions = scores.argmax(dim=1)
```

Hvilken form får `predictions`, og hva representerer tallene?

Bilder som tensorer

0.0	0.0	0.0	0.0
0.0	0.4	0.8	0.0
0.0	0.8	1.0	0.0
0.0	0.0	0.0	0.0

Én piksel = én lysstyrke.

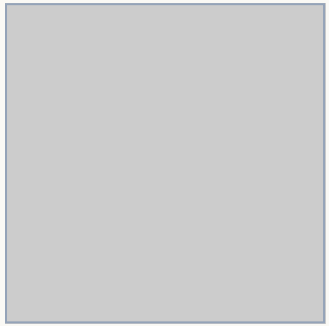
0 er svart. 1 er hvitt.

```
image[0, row, column]
```

Et gråtonebilde: `[1, H, W]`

Et RGB-bilde: `[3, H, W]`

Pikselverdier og skalering



204 / 255 = 0.800

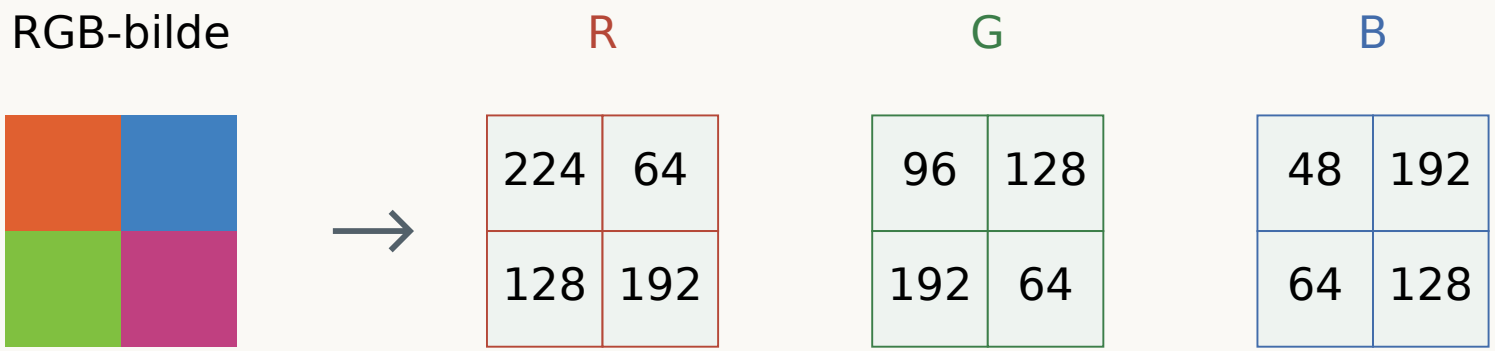
Pikselverdi:

0: svart · 255: hvitt

```
raw_pixels = torch.tensor([0, 128, 255], dtype=torch.uint8)
pixels = raw_pixels.float() / 255
# tensor([0.0000, 0.5020, 1.0000])
```

Skalering endrer tallområdet, men bevarer relative lysstyrker.

Fargekanaler



Tre kanaler, samme pikselposisjoner: [3, 2, 2]

```
red = image[0]    # [H, W]
green = image[1]  # [H, W]
blue = image[2]   # [H, W]
```

Fashion-MNIST har én gråtonekanal: `[1, 28, 28]`.

Bildeform og batchakse

Pikselmatrise `[28, 28]` → Én kanal `[1, 28, 28]` → 64 bilder `[64, 1, 28, 28]`

```
image.shape              # [1, 28, 28]
image.squeeze(0).shape   # [28, 28], for visning med imshow
```

Akse	Betydning
`B`	Antall bilder i batchen
`C`	Antall kanaler per bilde
`H, W`	Høyde og bredde i piksler

Fashion-MNIST

```python
from pytorch_101.utils import CLASS_NAMES, download_fashion_mnist

train_data, test_data = download_fashion_mnist()
image, label = train_data[0]
image.shape          # [1, 28, 28]
CLASS_NAMES[label]   # Navnet på riktig klasse
```

10 klasser · 20 000 treningsbilder · 1 000 testbilder

Klassefordeling og referansenøyaktighet

Et tenkt datasett med 100 bilder:

Klasse	Antall
T-skjorte	80
Bukse	10
Sko	10

En modell som alltid velger T-skjorte får **80 % nøyaktighet**, men finner ingen bukser eller sko.

```
baseline_accuracy = max(class_counts) / sum(class_counts)
```

Oppgaver 02 · Bilder

`oppgaver/02_bilder.ipynb`

Visualisering av bilder, klassefordeling og nøyaktighet ved flertallsgjetting.

Batcher og DataLoader

```
from torch.utils.data import DataLoader

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
images, labels = next(iter(train_loader))
images.shape  # [64, 1, 28, 28]
labels.shape  # [64]
```

Dataset henter eksempler. **DataLoader** samler dem i grupper.

`shuffle=True` stokker rekkefølgen for hver gjennomgang.

Lineær klassifikasjon

$$Z = XW + b$$

$$[B, 784] \mathbin{@} [784, 10] + [10] \longrightarrow [B, 10]$$

```
flat_images = images.flatten(start_dim=1)
weights = torch.randn(784, 10) * 0.01
bias = torch.zeros(10)
scores = flat_images @ weights + bias
```

Vekter bestemmer bidragene. **Bias** gir ett konstantledd per klasse.

nn.Linear og nn.Sequential

```
from torch import nn

model = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))
scores = model(images)              # [B, 10]
predictions = scores.argmax(dim=1) # [B]
```

`nn.Linear` eier vekter og bias. `nn.Sequential` setter lag i rekkefølge.

Skårene kalles **logits** og er ikke sannsynligheter. Predikert klasse er indeksen til den høyeste skåren.

Nøyaktighet = antall riktige / antall eksempler.

Oppgaver 03 · Prediksjoner

`oppgaver/03_prediksjoner.ipynb`

Vektede summer, klasseskårer og en lineær modell med `nn.Linear`.

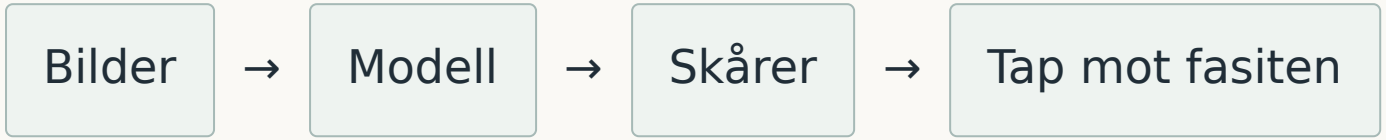
Kontrollspørsmål · Utflating

Et bilde med formen `[1, 28, 28]` flates ut til `[784]`.

Hva bevares, og hva er ikke lenger eksplisitt i formen?

Kan vi rekonstruere det opprinnelige bildet?

Trening av modellparametere



1. Beregn prediksjoner og tap (**forward**).
2. Beregn hvordan parameterne påvirker tapet (**backward**).
3. Juster vekter og bias (**optimizer**).

Treningen justerer parameterne for å redusere tapet på treningsdataene.

Kryssentropi og nøyaktighet

Fasit: klasse 1.

Sannsynligheter	Predikert klasse	Riktig?	Tap $-\log(p_y)$
`[0.4, 0.6]`	1	Ja	0.51
`[0.1, 0.9]`	1	Ja	0.11

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(scores, labels)
```

Send **rå skårer** `[B, 10]` og **klasseindekser** `[B]` inn i tapsfunksjonen.

Gradienter og læringsrate

Gradienten beskriver hvordan tapet endres ved små endringer i parameterne.

Enkel gradient descent:

$$w_{\text{ny}} = w - \eta \frac{\partial L}{\partial w}$$

Læringsraten η styrer størrelsen på steget.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Vi bruker **Adam**, som også tar hensyn til tidligere gradienter.

Én parameteroppdatering

```
optimizer.zero_grad()           # Nullstill gamle gradienter
scores = model(images)          # Beregn skårer
loss = loss_fn(scores, labels)  # Sammenlign med fasiten
loss.backward()                 # Beregn gradienter
optimizer.step()                # Oppdater vekter og bias
```

Batch: én gruppe bilder per oppdatering.

Epoke: én gjennomgang av treningsutvalget.

I oppgavene: batchstørrelse **64**, **25** epoker.

Evaluering på testdata

```
model.eval()
with torch.no_grad():
    scores = model(images)
    predictions = scores.argmax(dim=1)
    accuracy = (predictions == labels).float().mean().item()
```

Her er `images` og `labels` en **testbatch**. Hele testsettet må evalueres.

`eval()` velger evalueringsmodus. `no_grad()` slår av gradientregistrering.

Ingen parameteroppdateringer på testdata.

Oppgaver 04 · Trening

`oppgaver/04_lineaer_klassifikator.ipynb`

Treningsløkke, tapskurve og testnøyaktighet for den lineære modellen.

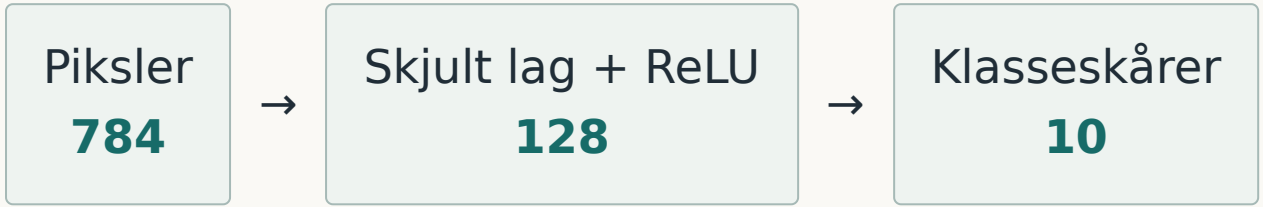
Kontrollspørsmål · Trening

```
loss.backward()
optimizer.step()
```

Hva er forskjellen mellom disse to kallene?

Hva beregnes, og når endres modellens parametere?

Nettverk med ett skjult lag




```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10),
)
```

Det skjulte laget lærer **mellomliggende representasjoner**.

Aktiveringsfunksjoner: ReLU

$$\mathrm{ReLU}(z) = \max(0, z)$$

```
values = torch.tensor([-2., -0.5, 0., 1., 3.])
torch.relu(values)  # tensor([0., 0., 0., 1., 3.])
```

ReLU lager et **knekkpunkt**: negative verdier blir null.

Flere lineære lag uten aktivering kan slås sammen til ett lineært lag.

Trening og sammenligning av modeller

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

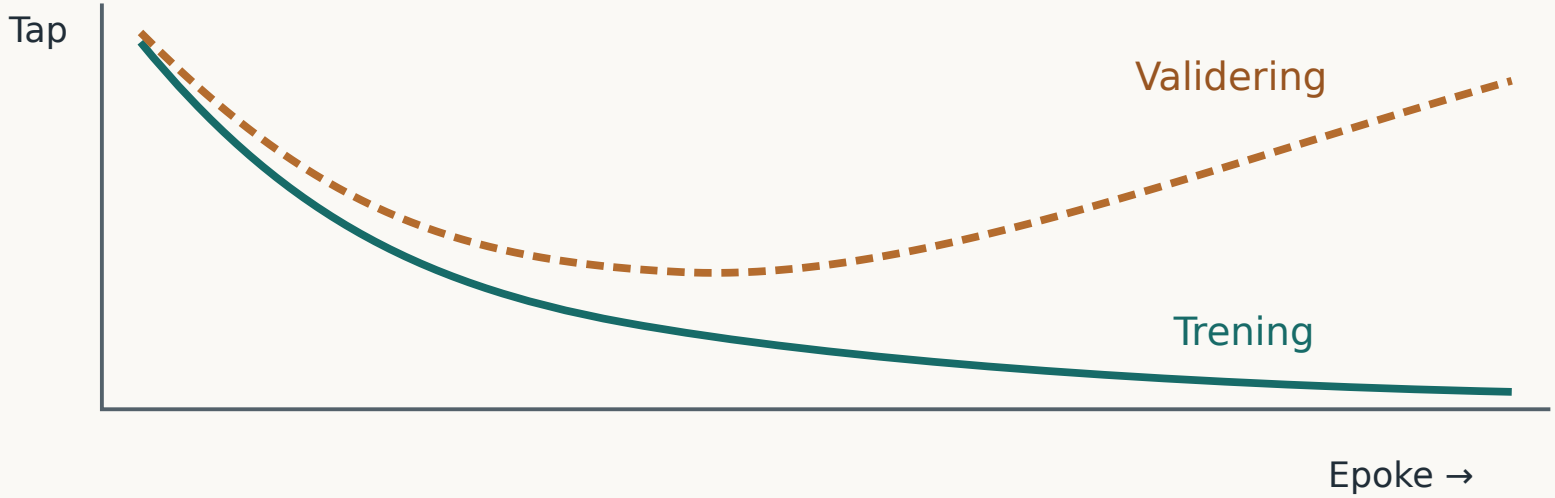
Opprett optimizer med **parameterne til den nye modellen**.

Treningsstegene og tapsfunksjonen er de samme.

Sammenlign med den lineære modellen:

- Nøyaktighet på usette bilder
- Treningstid
- Hvilke klasser modellen forveksler

Overtilpasning og generalisering



Trening: lær parametere · **Validering:** velg modell · **Test:** sluttvurdering

Skjematiske kurver. Kurset bruker faste hovedinnstillinger og separate trenings- og testutvalg.

Oppgaver 05 · Nevrale nettverk

`oppgaver/05_nevralt_nettverk.ipynb`

Trening av nettverket, sammenligning med den lineære modellen og feilanalyse med forvekslingsmatrise.

8 skjulte nevroner og prosjektet i `06_bonus.ipynb` er valgfrie.

Oppsummering

Del	PyTorch	Rolle
Data	`torch.Tensor`, `DataLoader`	Tall, former og batcher
Modell	`nn.Linear`, `nn.ReLU`	Inngang → skårer
Tap	`nn.CrossEntropyLoss`	Skårer og fasit → tap
Gradienter	`loss.backward()`	Deriver tapet med hensyn på parameterne
Oppdatering	`optimizer.step()`	Juster parameterne
Evaluering	`eval()`, `no_grad()`	Undersøk usette data